CS5013 Design Document

Brahmaputra Hostel Asset and Room Lifecycle Management System

Akhilesh K.R – CS26E001 Mysha Ali – CS26E008

11 September 2026

1 Architecture

The system will be implemented as a responsive mobile application. The backend will be implemented using Java and Spring Boot, with PostgreSQL used for persistent storage. The system will initially run locally for development and stakeholder testing, with deployment to an IIT Madras server or suitable hosted environment if access is provided.

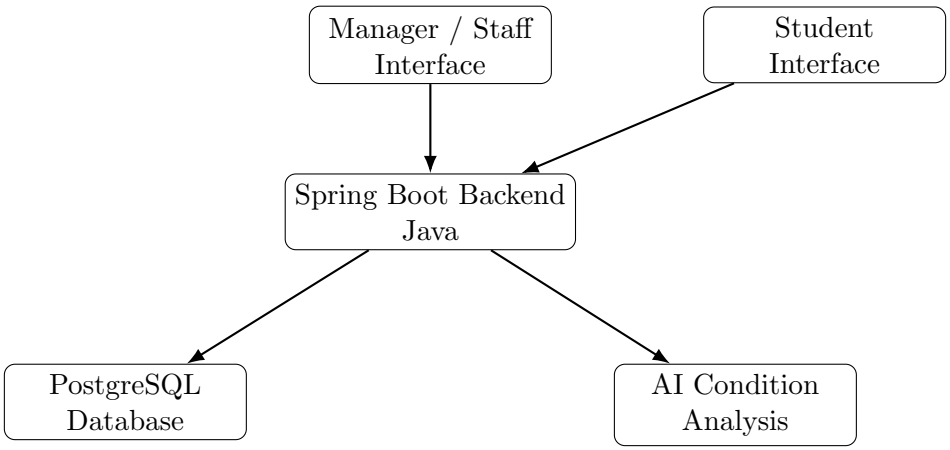


Figure 1: System architecture

1.1 Frontend

The frontend will use Flutter: A cross-platform mobile application providing tailored interfaces for Hostel Staff, the Hostel Manager, and Students. It handles UI rendering, local state management, and direct hardware interaction. The manager/staff interface will provide access to rooms, assets, inspections and asset history.

1.2 Backend

The backend will use Spring Boot and will expose REST APIs for:

- room and asset management,
- condition recording,
- inspections,
- asset history, and
- AI condition analysis.

1.3 Database

PostgreSQL will store the main entities required by the system:

- Users
- Rooms
- Occupancy records
- Assets
- Inspections
- Asset condition records
- Repair/history records

The room will be the central unit connecting occupants, assets and inspection records.

1.4 AI Component

The AI component will be used for condition analysis. It will analyse available inspection information to identify possible changes or anomalies in asset condition.

The core asset and inspection system will remain functional without the AI component, as the availability of sufficient historical data cannot be guaranteed.

2 Module Split

The system will be divided into four modules, with each team member responsible for two primary modules. Both members will jointly handle integration and testing.

1. Authentication and Access Control

- Manager/staff login
- Student login
- Role-based access control
- Restricting student access to their assigned room

2. Dashboard and Interface

- Recording room occupancy during inspections
- Periodic room inspections
- Before-occupancy and after-vacancy verification
- Recording inspection observations and condition updates

3. Asset Lifecycle Management

- Asset registry
- Association of assets with rooms
- Asset condition records
- Asset history and repair records

4. Analytics and AI

- Inspection and asset status dashboard
- Identification of repeatedly damaged assets
- Room-level issue summaries
- AI-based condition analysis
- Integration of AI-generated condition flags into the inspection workflow

Shared Work

Both members will jointly handle:

- Database design and integration
- Frontend-backend integration
- System integration
- Testing
- Stakeholder feedback
- Final deployment

3 Test Plan

Testing will use a synthetic Brahmaputra Hostel dataset representing single, double and triple-sharing rooms, occupants, room assets, condition records, inspections and repair histories. If actual de-identified data is provided by the stakeholder, it will be used where permitted.

3.1 Unit Tests

The following cases will be tested at the individual module level:

- A room cannot exceed its defined occupancy capacity.
- An asset cannot be assigned to multiple rooms at the same time.
- An inspection must reference an existing room.
- A condition record must reference an existing asset.
- Invalid condition values are rejected.
- Only authorized users can modify asset and inspection records.

3.2 Integration Tests

The interaction between modules will be tested using the following workflows:

- Assigning a student to a room updates the room's occupancy information.
- Assigning an asset to a room makes it available in the room's asset list.
- Recording an inspection creates corresponding condition records.
- A later inspection can retrieve the previous condition of an asset.
- A vacancy inspection records the final condition while preserving previous inspection history.
- Asset history displays condition and repair records in chronological order.

3.3 Acceptance Tests

The following tests correspond to the stakeholder-facing workflow.

Test	Scenario	Expected result
T1	Staff selects a Brahmaputra room and views its occupants and assets.	The correct room, occupancy and associated assets are displayed.
T2	Staff records the condition of assets in a room.	The condition is stored against the correct asset and room.
T3	A vacancy inspection records a changed or damaged asset.	The change is recorded and is visible in the asset history.
T4	A vacancy inspection identifies a missing asset.	The system records the discrepancy and marks the asset as requiring attention.

T5	A manager views an asset with repeated issues.	The manager can identify the repeated issue from the asset history without manually combining records.
T6	A student attempts to access another room.	The system denies access.
T7	AI analysis is performed on inspection information containing a known condition change.	The system produces a condition-analysis flag for staff review.

3.4 AI Evaluation

The AI component will be evaluated against staff-verified results where sufficient data is available.

For condition classification or anomaly detection, appropriate measures may include:

- Precision
- Recall
- F1 score
- False-positive rate

If sufficient real historical data is unavailable, the AI component will be evaluated using a documented synthetic dataset, while the core system will continue to be tested independently of AI.

4 Revised Milestone Plan

The milestone plan from the proposal is refined below into concrete development stages.

Date / Period	Milestone
12–18 Sep	Set up the Spring Boot backend, PostgreSQL database and responsive frontend. Implement basic authentication and user roles.
19–25 Sep	Implement room, occupancy and asset management. Populate the system with a synthetic Brahmaputra Hostel dataset.
26 Sep–2 Oct	Implement condition recording and periodic inspection workflows. Implement asset history.
3–8 Oct	Implement vacancy inspection and condition comparison. Integrate the core modules.
9 Oct 2026	**Mid-demo:** Demonstrate the core workflow end-to-end: asset assignment, condition recording and vacancy inspection.
10–18 Oct	Implement dashboard and analytics for inspection status, assets requiring attention and repeated issues.
19–26 Oct	Implement and evaluate the AI-based condition-analysis component using available real or synthetic data.
27 Oct–2 Nov	System integration, testing and stakeholder feedback. Fix issues identified during testing.
3–5 Nov	Finalize the application, deployment and documentation.
6 Nov 2026	**Final submission:** Stakeholder can run the complete system, including the integrated AI-based condition-analysis component.